

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

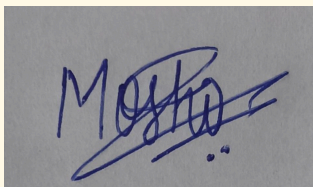
Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA0305	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Moshika M	Reg. No.:	192424064
Section / Batch:	B.Tech (AI-DS)	Date:	14.08.2026

Code of Conduct

I Moshika M - 192424064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the Student:

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

1. A company is developing an AI-powered customer support chatbot for banking services. The chatbot uses Context-Free Grammar (CFG) for parsing user queries. However, it faces issues such as:

Incorrect interpretation of ambiguous sentences

Failure to handle subject–verb agreement

Inefficient parsing for long queries

Example user query:

“Show me the transactions with the card from last month”

a) Analyze the ambiguity present in the given sentence using CFG

b) Evaluate the limitations of CFG in handling real-world conversational queries

c) Evaluate and propose an improved solution using PCFG, feature structures, or Earley parsing

d) Justify how your proposed method improves accuracy and efficiency in an industry setting

ANSWER:

1. AI-Powered Banking Customer Support Chatbot Using CFG

a) Analysis of Ambiguity Using CFG

The sentence “**Show me the transactions with the card from last month**” contains structural ambiguity. The phrase “**with the card**” can be interpreted in more than one way.

The first interpretation is that “**with the card**” describes the transactions. In this case, the user wants to see transactions that were made using a particular card.

[Show [me [the transactions [with the card] [from last month]]]]

Here, both *with the card* and *from last month* modify **transactions**.

A second interpretation is that “**with the card from last month**” refers to a particular card. The system may interpret the sentence as asking for transactions associated with a card that was obtained or used from last month.

A simplified CFG can represent the sentence as:

$S \rightarrow VP$

$VP \rightarrow V\ NP\ PP\ PP$

$NP \rightarrow Det\ N$

$PP \rightarrow P\ NP$

The grammar can generate several syntactically valid structures without necessarily identifying the user's intended meaning. CFG mainly describes whether a sentence is structurally valid; it does not provide sufficient contextual information to select the correct interpretation.

Therefore, the banking chatbot may incorrectly retrieve transactions, particularly when several cards or accounts are associated with the customer.

b) Limitations of CFG for Real-World Conversational Queries

Although CFG is useful for representing basic grammatical structures, it has several limitations in conversational banking applications.

1. **Ambiguity:** CFG can generate multiple valid parse trees but cannot reliably determine which interpretation is intended.
2. **No probability:** All valid grammatical structures are essentially treated equally, so the system cannot rank the most likely interpretation.
3. **Limited contextual understanding:** CFG does not naturally consider previous conversation, user history, or banking context.
4. **Subject–verb agreement:** Basic CFG rules do not automatically ensure that subjects and verbs agree in number or person.
5. **Long sentences:** The number of possible parse structures can increase significantly as sentence length and complexity increase.
6. **Conversational language:** Users may provide incomplete queries such as “*transactions last month*” or “*show card payments*”, which may not conform to complete CFG structures.
7. **Domain-specific expressions:** Banking terms, abbreviations, and informal expressions require additional grammar rules.

Hence, a basic CFG alone is insufficient for a robust banking chatbot.

c) Improved Solution Using PCFG, Feature Structures, and Earley Parsing

A better solution is to combine **PCFG, feature structures, and Earley parsing**.

PCFG assigns probabilities to grammar rules. For example, if historical banking conversations show that “*with the card*” usually modifies “*transactions*”, the corresponding parse tree can receive a higher probability.

Feature structures can store grammatical information such as:

- Number: singular/plural
- Person: first/second/third
- Tense
- Case
- Agreement features

For example:

NP [number = plural] → transactions

VP [number = plural] → show

The parser can reject or penalize structures that violate agreement constraints.

Earley parsing is particularly useful because it can parse arbitrary CFGs and handle ambiguous and incomplete inputs efficiently. It can maintain multiple possible interpretations while processing the query rather than repeatedly starting the parsing process from the beginning.

A practical architecture would therefore be:

User Query → Tokenization → Feature Extraction → Earley Parser + PCFG → Best Parse → Intent Extraction → Banking API → Response

d) Improvement in Accuracy and Efficiency

The proposed approach improves accuracy because PCFG uses probabilities learned from real customer queries to select the most likely interpretation. Feature structures ensure grammatical agreement and provide additional linguistic constraints.

Earley parsing improves efficiency when the grammar is complex and the input is ambiguous or incomplete. It also supports incremental parsing, which is useful for conversational systems.

For example:

“Show me the transactions with the card from last month”

can be converted into a structured intent such as:

Intent: Transaction_Search

Object: Card Transactions

Time: Last Month

Customer: Current User

The banking system can then retrieve the appropriate transactions.

In an industry setting, the combination reduces incorrect transaction retrieval, improves response reliability, supports natural conversational input, and provides a scalable foundation for banking customer-support applications.

2. A voice assistant system (like Alexa/Siri-type applications) processes spoken commands using parsing techniques. The system currently uses top-down parsing, but faces:

Backtracking issues

Poor handling of incomplete or ambiguous input

Difficulty in real-time response

Example command:

“Book a flight to Delhi with a window seat”

a) Analyze how the given sentence can lead to multiple parse structures

b) Analyze the limitations of top-down parsing in this real-time application

c) Analyze how Earley parsing can improve handling of ambiguity and partial inputs

d) Compare and analyze the performance of top-down vs Earley parsing for such industry use cases

ANSWER:

a) Multiple Parse Structures for the Given Sentence

The command **“Book a flight to Delhi with a window seat”** can produce different interpretations depending on how the prepositional phrase **“with a window seat”** is attached.

One interpretation is:

Book [a flight to Delhi] [with a window seat]

Here, *with a window seat* specifies the user's preferred seat.

Another possible interpretation is:

Book [a flight [to Delhi with a window seat]]

In this structure, the parser may incorrectly associate the phrase *with a window seat* more closely with the flight destination phrase.

A simplified CFG could contain rules such as:

$S \rightarrow VP$

$VP \rightarrow V NP$

$NP \rightarrow Det N PP$

$PP \rightarrow P NP$

Because CFG rules can generate multiple valid structures, the parser may need additional information to determine the intended interpretation.

In a voice assistant, this ambiguity is important because the system must convert the user's command into a specific action:

Intent: Book Flight

Destination: Delhi

Seat Preference: Window

b) Limitations of Top-Down Parsing

Top-down parsing begins with the start symbol and attempts to derive the input sentence by expanding grammar rules.

For a real-time voice assistant, this approach has several limitations:

1. **Backtracking:** When an attempted derivation fails, the parser may need to return to an earlier decision and try another rule.
2. **Repeated work:** Similar grammar alternatives may be explored multiple times.
3. **Ambiguous input:** Several possible structures may require extensive exploration before the correct interpretation is selected.
4. **Incomplete speech:** Voice commands may stop unexpectedly or contain fragments such as *“Book a flight to...”*.
5. **Real-time constraints:** Excessive backtracking can increase response latency.
6. **Speech-recognition errors:** Spoken language may contain repetitions, corrections, or missing words that do not match expected grammar structures.

For a voice assistant, even a small delay can negatively affect the user experience.

c) How Earley Parsing Improves Ambiguity and Partial Input Handling

Earley parsing is a dynamic-programming parsing technique capable of handling arbitrary context-free grammars. Instead of repeatedly exploring the same possibilities, it stores intermediate parsing states.

The three major operations are:

Prediction → Scanning → Completion

For the command:

“Book a flight to Delhi with a window seat”

the parser can maintain multiple possible interpretations while processing the sentence. When additional words are received, the parser can update the existing states rather than completely restarting.

Earley parsing is also useful for incomplete input. For example:

“Book a flight to...”

The parser can determine that a destination is expected and wait for additional input.

This is particularly valuable in speech-based systems because speech arrives sequentially rather than always as a complete sentence.

d) Top-Down vs Earley Parsing

Feature	Top-Down Parsing	Earley Parsing
Parsing direction	Start symbol to input	Dynamic chart-based parsing
Ambiguity handling	May require extensive backtracking	Maintains multiple possible parses
Backtracking	Common	Greatly reduced
Incomplete input	Limited	Better suited
Arbitrary CFG	Depends on implementation	Supported
Long/complex sentences	Can become inefficient	More efficient through chart parsing
Real-time suitability	Lower for highly ambiguous input	Better
Memory usage	Usually lower	Higher due to chart storage
Voice-assistant use	Suitable for simple commands	Suitable for complex and ambiguous commands

For simple commands such as **“Call Mom”**, top-down parsing may be fast and sufficient. However, commands involving multiple phrases, optional information, ambiguity, or incomplete speech are better handled using Earley parsing.

Therefore, an industry voice assistant can use a hybrid strategy: a lightweight parser for simple commands and Earley-based parsing for complex or ambiguous commands. This provides a balance between speed, accuracy, and robustness.

Overall, Earley parsing is better suited to sophisticated voice assistants because it can preserve alternative interpretations, handle incomplete input, and avoid unnecessary repeated parsing.

3. A healthcare company is developing an AI system to analyze patient medical reports and automatically extract critical information such as diagnosis, prescribed treatments, and follow-up actions.

The system must handle:

Complex and lengthy medical sentences

Ambiguity in sentence structure

Accurate subject-verb agreement

Relationships between medical actions and entities (sub-categorization)

Real-time processing for hospital use

Example input:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

The current system, based only on basic Context-Free Grammar (CFG), fails to correctly interpret relationships and produces inconsistent outputs.

a) Design a complete NLP architecture integrating: CFG for syntactic structure Probabilistic CFG (PCFG) for ambiguity resolution Feature Structures for agreement handling Efficient parsing techniques

b) Develop a step-by-step workflow showing how the system processes the given medical sentence from input to structured output (diagnosis, action, location).

c) Create a method to: Incorporate sub-categorization frames for medical verbs Handle real-time data processing Improve accuracy and scalability in a hospital environment

ANSWER:

a) Complete NLP Architecture

A healthcare NLP system must combine multiple linguistic techniques because basic CFG alone cannot adequately handle complex medical language.

CFG Layer

CFG provides the basic syntactic structure of the medical sentence. It identifies relationships between nouns, verbs, phrases, clauses, and prepositional phrases.

For example:

$S \rightarrow NP VP$

The sentence can be separated into a subject and predicate, after which the relative clause and coordinated actions can be identified.

PCFG Layer

The sentence may have multiple possible parse structures. PCFG assigns probabilities to grammar rules based on training data and selects the most probable interpretation.

This is useful for resolving ambiguities involving phrases such as:

“who reviewed the patient last week”

and:

“in Chennai.”

Feature Structures

Feature structures represent grammatical information such as:

Number = singular

Person = third

Tense = present

For the sentence:

“The doctor ... recommends...”

the subject *doctor* is singular, so the verb *recommends* must also be singular.

Feature unification prevents incorrect combinations such as:

Doctor recommend

when the correct form is:

Doctor recommends

Efficient Parsing

An Earley parser or another chart-based parser can store partial results and avoid repeatedly parsing the same sentence fragments. This makes it suitable for complex medical sentences and real-time hospital systems.

b) Step-by-Step Workflow

Step 1: Input Processing

The system receives:

“The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

The sentence is normalized and divided into tokens.

Step 2: Named Entity Recognition

The system identifies important entities:

- **Doctor** → **Medical professional**
- **Patient** → **Patient entity**
- **Medication** → **Treatment**
- **Chennai** → **Location**

Step 3: Syntactic Parsing

CFG identifies the major structure:

Subject: The doctor who reviewed the patient last week

Main Verb: recommends

Actions: starting medication and scheduling a follow-up visit

The relative clause:

“who reviewed the patient last week”

is associated with **doctor**.

Step 4: Feature Structure Analysis

The parser identifies:

Doctor → **Singular, Third Person**

Therefore:

recommends → **Singular, Third Person**

The system also identifies the appropriate tense and grammatical relationships.

Step 5: PCFG-Based Disambiguation

The PCFG assigns probabilities to alternative parse structures and selects the interpretation that best matches learned medical-language patterns.

The system determines that **“in Chennai”** most naturally identifies the location of the follow-up visit rather than the location of the medication.

Step 6: Sub-Categorization Analysis

The system uses medical verb frames.

For example:

recommend + action

A medical professional can recommend an action such as:

recommend starting medication

or:

recommend scheduling a follow-up visit

The verb **review** expects an entity representing the object being reviewed:

review(Doctor, Patient)

Step 7: Semantic Information Extraction

The system extracts:

Doctor → recommends → starting medication

Doctor → recommends → scheduling follow-up visit

Follow-up visit → Location → Chennai

The relative clause also establishes:

Doctor → reviewed → Patient

Time → Last week

Step 8: Structured Output

The final output can be represented as:

Diagnosis: Not explicitly stated in the sentence

Medical Action 1: Start medication

Medical Action 2: Schedule follow-up visit

Location: Chennai

Reviewer: Doctor

Reviewed Entity: Patient

Review Time: Last week

This is important because the system should not invent a diagnosis when the source sentence does not explicitly provide one.

c) Method for Sub-Categorization, Real-Time Processing, Accuracy and Scalability

1. Incorporating Sub-Categorization Frames

Medical verbs should be associated with expected argument structures.

Examples include:

prescribe → Doctor + Medication + Patient

recommend → Doctor + Treatment/Action

diagnose → Doctor + Disease/Condition + Patient

review → Doctor + Medical Record/Patient

schedule → Staff/System + Appointment

These frames help the system determine which entities can logically participate in an action.

For example:

“The doctor recommends starting medication.”

matches:

recommend(Agent, Action)

while an unlikely structure can be rejected or given a lower probability.

2. Real-Time Data Processing

For hospital environments, the system should use an incremental processing architecture.

Incoming Report → Streaming Preprocessing → Incremental Parsing → Entity Extraction → Semantic Analysis → Structured Clinical Record

The parser should process sentences as they arrive instead of waiting for an entire large document. Frequently used grammar rules and medical terminology can also be cached to reduce processing time.

3. Improving Accuracy

Accuracy can be improved by combining:

- Medical-domain PCFG models.
- Feature structures for agreement.
- Clinical Named Entity Recognition.
- Medical sub-categorization frames.
- Medical terminology databases.
- Context-aware language models.
- Confidence scoring and validation.
- Human review for low-confidence outputs.

The system should also distinguish between **explicit information** and **inferred information** to avoid generating unsupported diagnoses or treatments.

4. Improving Scalability

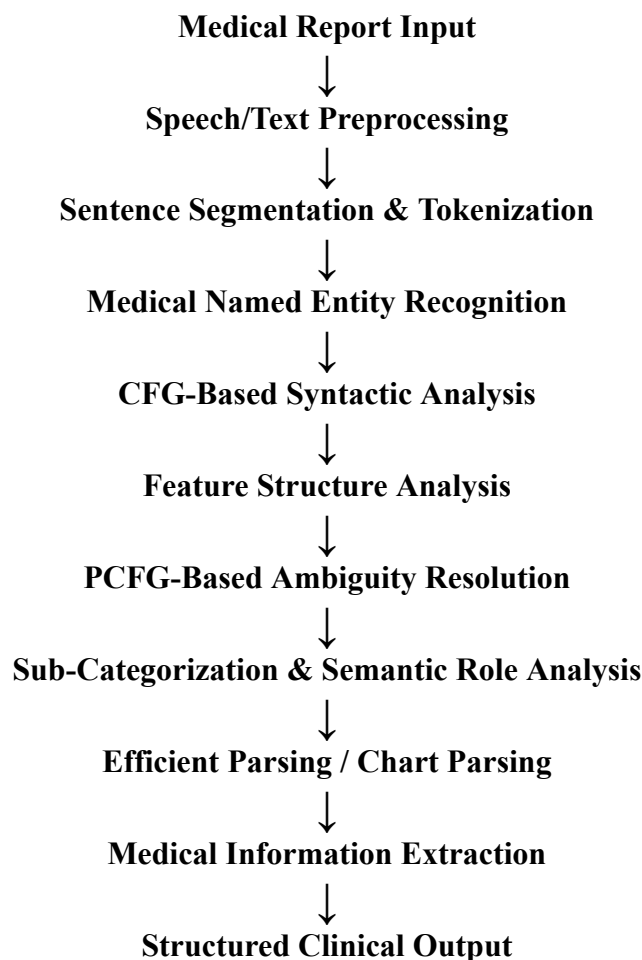
A hospital may process thousands of reports simultaneously. Therefore, the NLP system should use modular services that can be independently scaled.

For example:

Preprocessing Service → Parsing Service → Medical NER Service → Semantic Analysis Service → Clinical Database

Multiple instances of computationally intensive services can run simultaneously. Frequently used medical vocabulary, grammar rules, and parsing results can be cached.

Proposed Architecture



The proposed architecture overcomes the limitations of basic CFG by combining syntactic rules, probabilistic reasoning, grammatical features, efficient parsing, and medical-domain knowledge. This allows the healthcare NLP system to process complex sentences more accurately while remaining suitable for real-time hospital applications.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1								
2								
3								

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____